

Realtime Streaming Image Processing on FPGA

Implementation Report Revision: AXI4-Stream Pipeline, Control FSM, and Verification



Lukas Roess, Valentin Bumeder, Jan Duchscherer, Justin Loeber

Embedded Systems, Academic Year 2025-2026

Revision date: 2026-02-22

1. Abstract

This revision aligns the implementation report with the current repository state and the TODO requirements for component-level technical depth. The active processing chain covered in this version is RGB -> Grayscale -> 3x3 line-buffer/window -> Sobel -> Frame compositor, with runtime control handled by the debounced click-detector path.

The report focuses on repository-owned assets under `rtl/`, `testbench/`, and `docs/report/analysis/`. In particular, this draft adds protocol background, per-component interface contracts, waveform-backed transaction timing, and synthesis evidence extracted from Vivado reports into CSV/JSON that are directly consumed by Typst in this document.

Standards-oriented claims for AXI4-Stream Video semantics and cocotb timing/driver behavior are tied to primary references from AMD, cocotb, and Digilent.[1], [2], [3], [4]

2. Scope and Revision Intent

2.1. In scope

- RTL modules and wrappers in `rtl/` for grayscale conversion, blur/sobel processing, compositing, and pipeline integration.
- cocotb verification framework in `testbench/`, including reusable AXI stream drivers/monitors and target-based execution.
- AXI4-Stream framing correctness, backpressure behavior, and frame-boundary control semantics.
- Vivado-backed synthesis/implementation utilization evidence for key modules and full-system placement.

2.2. Out of scope

- Board-level timing closure sign-off and full hardware performance profiling.
- Non-repository software UX flows.
- Legacy FAST-specific implementation details that are not part of the current active integration branch.

2.3. Draft corrections applied

Compared to previous draft snapshots, this revision:

- removes stale naming and ownership claims around deprecated wrappers,
- centers integration ownership on `FRAME_COMPOSITOR` and the current pipeline wrapper,
- replaces stale aggregate testcase statements with command-backed evidence,
- adds explicit notes where regressions/noise remain open rather than implying full closure.

3. AXI4-Stream Video Protocol Background

The stream contract used by RTL and cocotb scoreboards is:

$$\text{transfer}_k = \text{tvalid}_k \wedge \text{tready}_k \quad (1)$$

For AXI4-Stream Video, UG934 imposes project-critical rules for active video beats:[1]

- TUSER[0] marks start-of-frame (SOF).
- TLAST marks end-of-line (EOL).
- While stalled (TVALID=1, TREADY=0), payload and sidebands remain stable.
- Only active pixels are transferred on the AXI stream.

Signal/Rule	Meaning	Project consequence
TUSER[0]	SOF marker	Control/state latching is performed at frame boundaries.
TLAST	EOL marker	Line-level alignment is preserved across RGB and grayscale branches.
TVALID && TREADY	Beat acceptance	Delay chains advance only on accepted beats.
stall stability	No payload mutation while blocked	Testbench includes backpressure patterns to detect illegal changes.

Table 1: AXI4-Stream video rules applied across this repository.

Project-specific stream order is consistent across RTL and Python models:

- wire-level RGB24 order is TDATA[23:0] = R|B|G,
- Python tuples and scoreboard comparisons use (R, G, B) after decode.

Platform context follows Digilent Zybo Z7 and Pcam 5C integration guidance for camera/display infrastructure.[4], [5]

4. Component Deep Dive: RGB_TO_GRAYSCALE

Component owner(s): Jan Duchscherer

4.1. Conceptual introduction

AXI_RgbToGrayscale ([rtl/RGB_TO_GRAYSCALE/hdl/axi_rgb_to_grayscale.vhd](#)) wraps the grayscale core E_RgbToGrayscale ([rtl/RGB_TO_GRAYSCALE/hdl/rgb_to_grayscale.vhd](#)) and emits two synchronized output branches: a grayscale stream (gray8) and an RGB stream (rbg888) that can either forward the original pixel or replicate grayscale data, depending on i_pass_through.

4.2. Vivado interface view

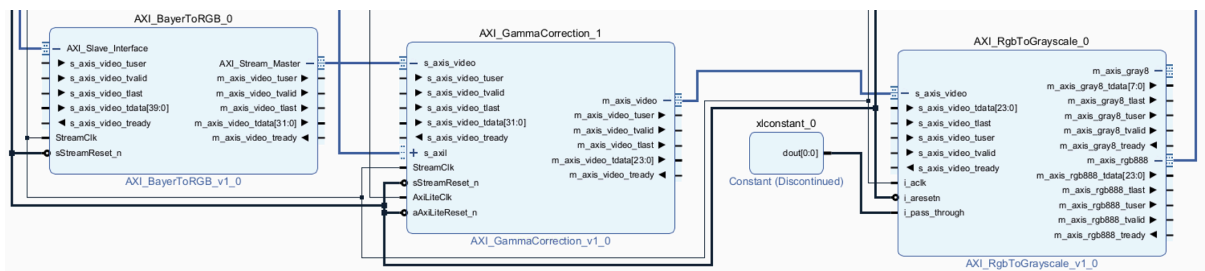


Figure 1: Vivado block context showing AXI_RgbToGrayscale insertion and interfaces.

4.3. Interface ports and generics

Signal/group	Dir.	Width	Purpose
G_COMPONENT_WIDTH	generic	default 8	Per-channel component width.
i_aclk, i_aresetn	in	1	Clock/reset for AXI stream logic.
i_pass_through	in	1	Selects original RGB forwarding versus grayscale-replicated RGB.
s_axis_video_*	in/out	AXI video	Input AXI4-Stream video channel (R B G payload order).
m_axis_rbg888_*	out/in	AXI video	RGB/RBG output branch for pipeline base stream.
m_axis_gray8_*	out/in	AXI video	Gray8 output branch for filter pipeline.

Table 2: AXI_RgbToGrayscale interface overview from [rtl/RGB_TO_GRAYSCALE/hdl/axi_rgb_to_grayscale.vhd](#).

4.4. Conversion formulas and implementation tradeoff

A common luminance-weighted floating-point model is:

$$Y_{\text{float}} = 0.299R + 0.587G + 0.114B \quad (2)$$

A fixed-point integer approximation suitable for 8-bit hardware is:

$$Y_{\text{fix8}} = \frac{77R + 150G + 29B + 128}{256} \quad (3)$$

The implemented RTL uses a shift/add approximation:

$$Y_{\text{rtl}} \approx \left(\frac{R}{4}\right) + \left(\frac{G}{2}\right) + \left(\frac{B}{4}\right) \quad (4)$$

This avoids multipliers and keeps the stage fully combinational (E_RgbToGrayscale), while AXI_RgbToGrayscale handles dual-branch handshake decoupling with a pending-beat register.

4.5. Typical transaction sequence

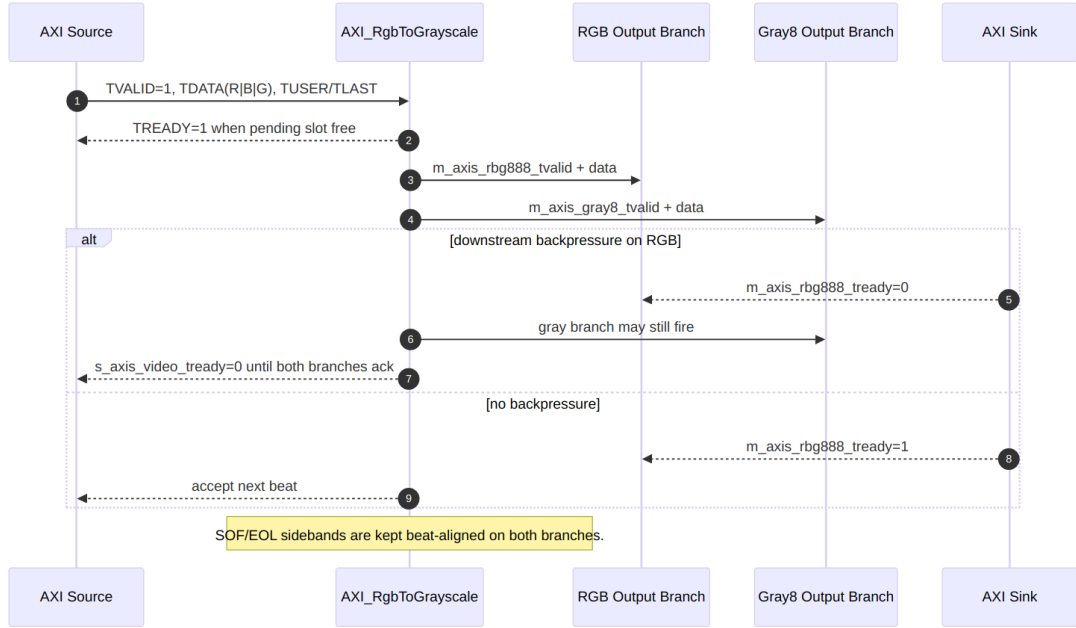


Figure 2: Typical dual-branch transaction for AXI_RgbToGrayscale under optional RGB-branch backpressure.

4.6. Timing diagram from cocotb VCD/GHW run

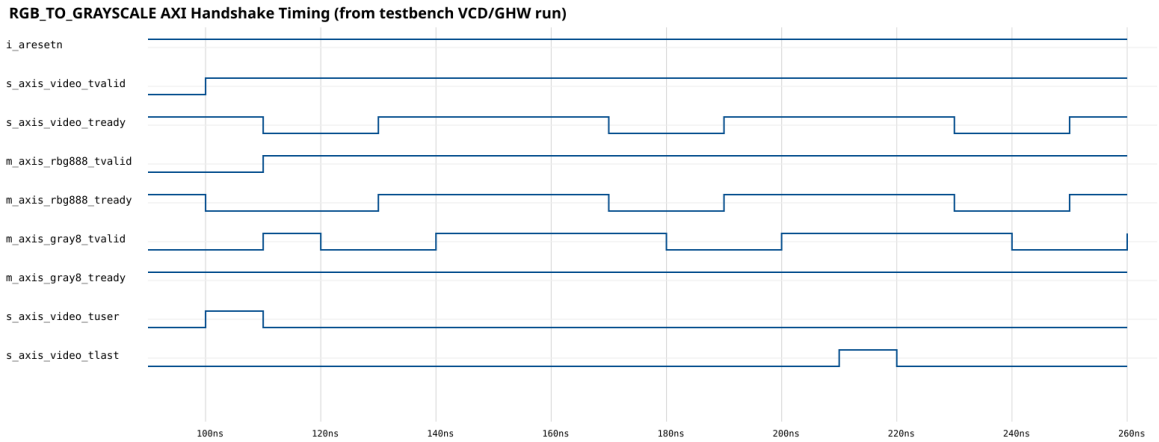


Figure 3: Measured AXI handshake timing for RGB_TO_GRAYSCALE from testbench VCD extraction.

Key observations from the waveform include:

- accepted input beats only when the internal slot is free,
- branch-level backpressure propagation to s_axis_video_tready,

- retained SOF/EOL alignment between RGB and gray branches.

5. Component Deep Dive: CLICK_DETECTOR Control FSM

Component owner(s): Valentin Bumeder, Jan Duchscherer

5.1. Conceptual introduction

The control path consists of DebouncedClickDetector ([rtl/CLICK_DETECTOR/hdl/debounced_click_detector.vhd](#)) and ClickDetector ([rtl/CLICK_DETECTOR/hdl/click_detection.vhd](#)). BTN1 cycles processing stages, while BTN2 cycles base-image behavior.

5.2. Vivado interface view

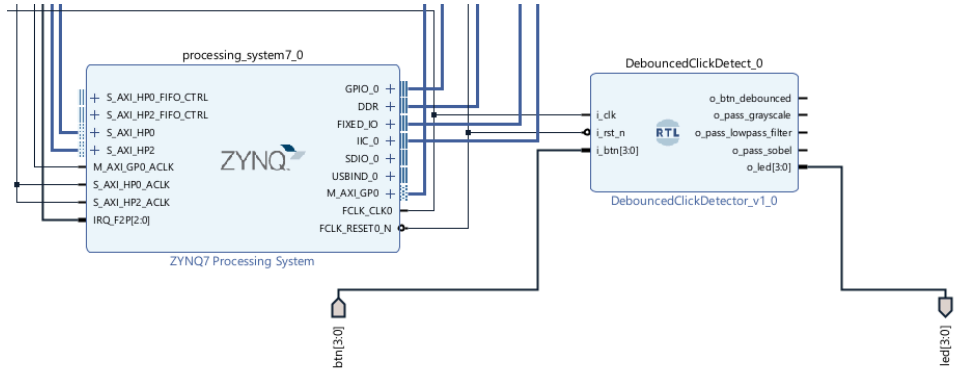


Figure 4: Vivado integration view showing debounced click-detector control IP in the processing system context.

5.3. Interface ports and generics

Signal/group	Dir.	Width	Purpose
G_CLK_FREQ_HZ, G_DEBOUNCE_NS	generic	integer	Debounce timing configuration in DebouncedClickDetector.
i_btn[3:0]	in	4	Physical button inputs before debounce.
o_btn_debounced, o_btn2_debounced	out	1 each	Edge-detected FSM trigger sources after debounce.
o_pass_grayscale	out	1	Base-stream selection: RGB passthrough vs gray replication.
o_pass_blurr_filter, o_pass_sobel	out	1 each	Processing-mode bypass controls.
o_overlay_zeros	out	1	Force binary-only overlay output mode.
o_led[3:0]	out	4	Runtime mode indicators.

Table 3: Click-detector control interfaces from [debounced_click_detector.vhd](#) and [click_detection.vhd](#).

5.4. FSM style classification (Mealy vs Moore)

Both FSM partitions in ClickDetector are Moore-style with respect to output behavior: output controls are derived from current state registers (s_current_state, s_base_current_state), while button edges only affect next-state transitions.

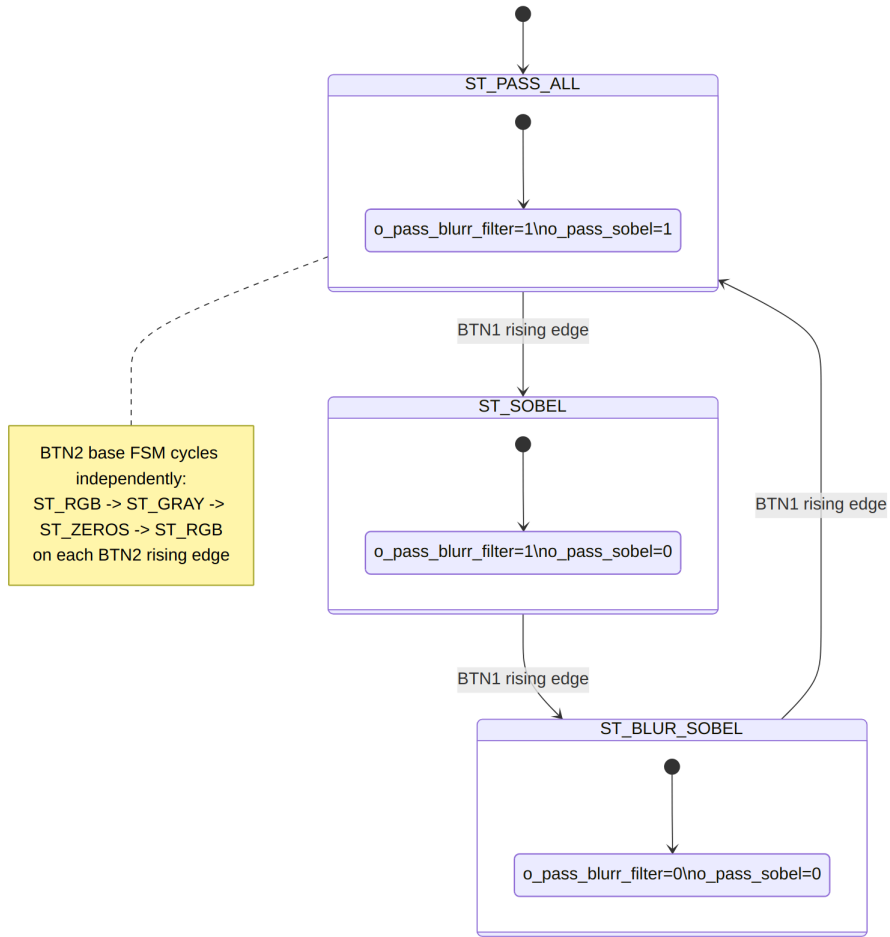


Figure 5: BTN1 processing-state FSM with output mode mapping and BTN2 independence note.

FSM	State	o_pass_blurr_filter	o_pass_sobel	Base/overlay outcome
processing	ST_PASS_ALL	1	1	Bypass blur and sobel; upstream grayscale stage remains available.
processing	ST_SOBEL	1	0	Sobel active without blur pre-stage.
processing	ST_BLUR_SOBEL	0	0	Blur + Sobel cascade active.
base	ST_RGB	-	-	o_pass_grayscale=1, o_overlay_zeros=0: RGB base shown.
base	ST_GRAY	-	-	o_pass_grayscale=0, o_overlay_zeros=0: gray-replicated base shown.
base	ST_ZEROS	-	-	o_pass_grayscale=0, o_overlay_zeros=1: base suppressed, binary overlay-only view.

Table 4: State/output matrix for processing and base-image FSM partitions.

6. Component Deep Dive: FRAME_COMPOSITOR

Component owner(s): Valentin Bumeder, Justin Loeber

6.1. Conceptual introduction

AXI_FrameCompositor ([rtl/FRAME_COMPOSITOR/hdl/axi_frame_compositor.vhd](#)) is the output owner for final RGB888 stream selection. It synchronizes delayed base RGB data with gray/mask timing and decides between binary-mask output or overlay compositing through FrameCompositor ([rtl/FRAME_COMPOSITOR/hdl/frame_compositor.vhd](#)).

6.2. Vivado interface view

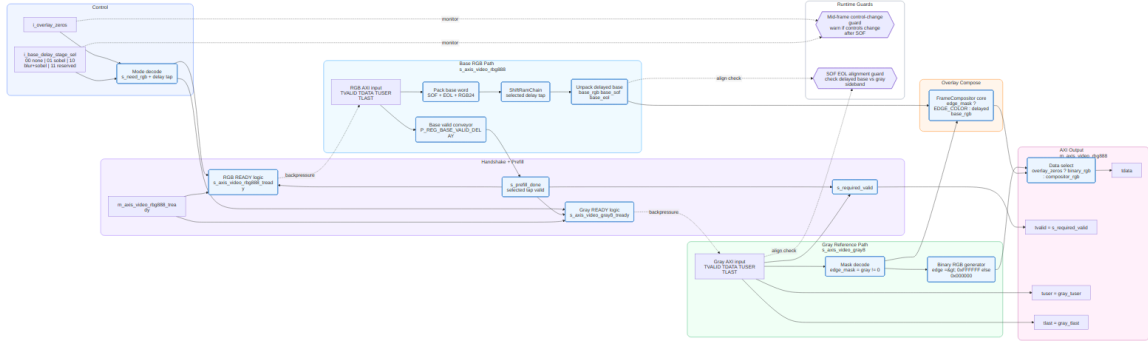


Figure 6: Frame compositor architecture and signal path used for delayed-base alignment and output ownership.

6.3. Interface ports and generics

Signal/group	Dir.	Width	Purpose
G_LINE_WIDTH, G_SOBEL_KERNEL_SIZE, G_BLUR_KERNEL_SIZE	generic	positive	Delay derivation parameters.
G_SOBEL_DELAY_OVERRIDE, G_BLUR_SOBEL_DELAY_OVERRIDE	generic	natural	Optional manual tap override.
i_overlay_zeros	in	1	Select binary-only output (1) or overlay compose (0).
i_base_delay_stage_sel[1:0]	in	2	Tap selector (00 none, 01 sobel, 10 blur+sobel, 11 reserved-alias).
s_axis_video_rbg888_*	in/out	AXI video	Base stream input used for delayed RGB alignment.
s_axis_video_gray8_*	in/out	AXI video	Gray/mask reference stream driving output timing.
m_axis_video_rbg888_*	out/in	AXI video	Composited output stream.

Table 5: AXI_FrameCompositor interface and control contract from [rtl/FRAME_COMPOSITOR/hdl/axi_frame_compositor.vhd](#).

6.4. Delay theory for stream synchronization

For odd kernel sizes, warm-up delay in accepted beats is modeled as:

$$D_{\text{stage}(W,K)} = (W + 1) \left(\frac{K - 1}{2} \right) \quad (5)$$

For this pipeline, Sobel and Blur+Sobel taps are:

$$D_{\text{sobel}} = D_{\text{stage}(W, K_{\text{sobel}})} \quad (6)$$

$$D_{\text{blur+sobel}} = D_{\text{stage}(W, K_{\text{blur}})} + D_{\text{stage}(W, K_{\text{sobel}})} \quad (7)$$

Because delays are chunked over 1024-beat `c_shift_ram_0` segments, effective tap latency includes segment-boundary overhead:

$$D_{\text{effective}} = D_{\text{requested}} + (N_{\text{chunks}} - 1) \quad (8)$$

This delay model is reflected by `ShiftRamChain` ([rtl/FRAME_COMPOSITOR/hdl/shift_ram_chain.vhd](#)) and consumed by `AXI_FrameCompositor` prefill gating (`s_prefill_done`, `s_base_delayed_valid`).

6.5. Typical transaction sequence

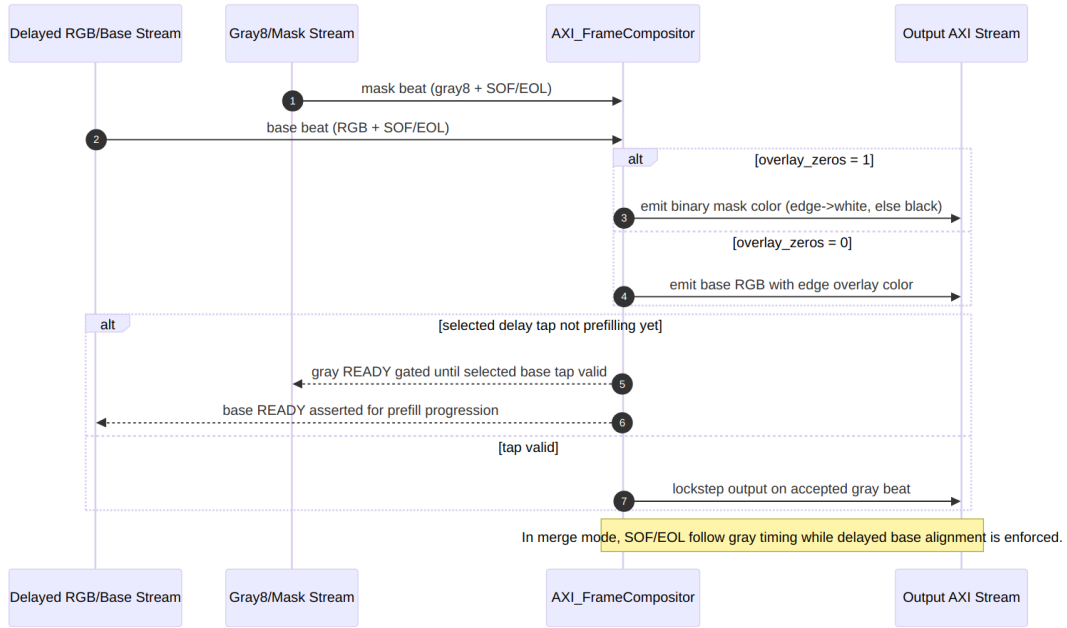


Figure 7: Representative AXI_FrameCompositor transaction showing mode-dependent output and prefill gating.

6.6. Timing diagram from cocotb VCD/GHW run

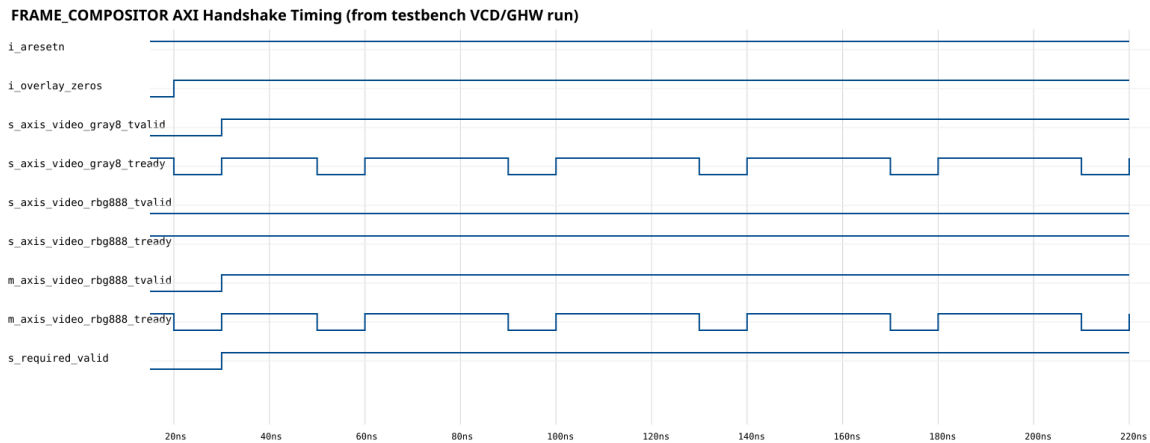


Figure 8: Measured FRAME_COMPOSITOR handshake behavior under periodic gray-stream backpressure.

Waveform interpretation confirms:

- merge mode waits for selected delayed-base tap validity before full lockstep emission,
- gray stream remains the output timing reference,
- READY behavior prevents deadlock during delay prefill while preserving beat-domain alignment.

7. Simulation Framework using cocotb and cocotbext-axi

Component owner(s): Lukas Roess, Jan Duchscherer, Valentin Bumeder, Justin Loeber

The verification stack is organized as reusable source/sink/model/scoreboard layers in `testbench/` and executed through target registration in `testbench/targets.toml`, with runner entrypoint `testbench/sim/run.py`.

Stimulus	DUT	Checks
drivers/axis*_source.py	RTL toplevel under tb-sim --target ...	monitors/axis*_sink.py + verification/scoreboard.py
custom serialization to project AXI wire order	AXI4-Stream video handshake + control	golden reference comparison and protocol assertions

Table 6: Testbench structure: Source (+ serialization) -> UUT -> Sink (+ deserialization) with scoreboard checks.

Framework elements aligned with cocotb timing/writing guidance:[2], [3], [6]

- deterministic reset/startup helper (common/reset.py),
- configurable backpressure generation (common/pause.py),
- typed AXI stream drivers and sinks for RGB and gray paths,
- per-target artifact generation (results.xml, .ghw/.vcd, output images).

Stress dimensions covered by active targets include:

- backpressure/stall robustness,
- reset and initialization transitions,
- frame-boundary correctness (SOF/EOL),
- mode/control transitions through button-driven FSM paths,
- timeout handling via cocotb with_timeout wrappers in sink APIs.

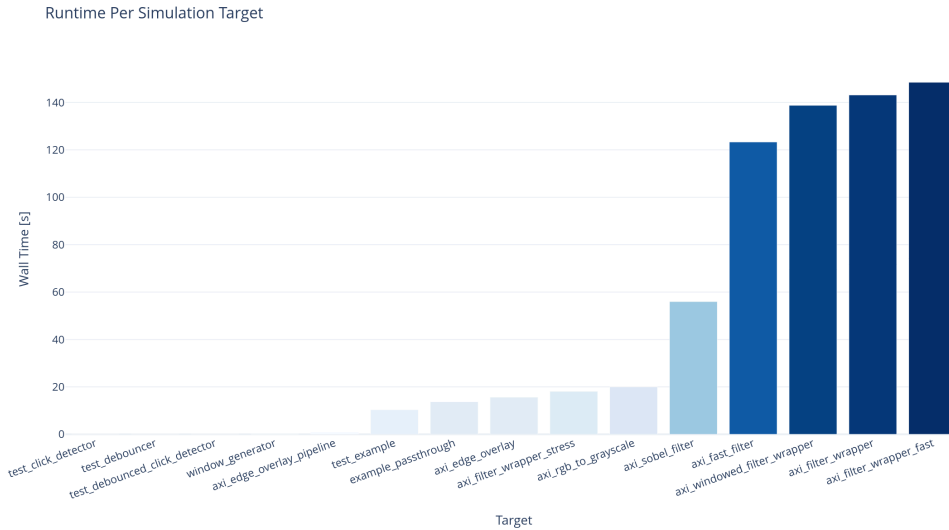


Figure 9: Stored runtime distribution by simulation target.

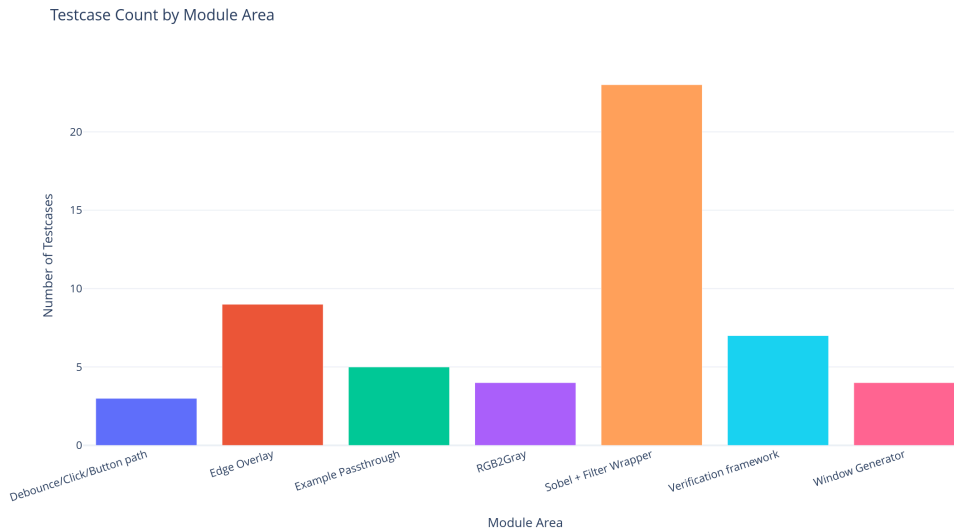


Figure 10: Stored testcase distribution by module area.

8. Synthesis and Implementation Results

Component owner(s): Engineering review

This section uses Typst file I/O (`csv(...)`) over cleaned artifacts generated from Vivado reports in `docs/report/data/`.

Module	LUT	FF	BRAM	DSP	Evidence source
RGB_TO_GRAYSCALE (AXI OOC)	48.0	30.0	0.0	0.0	docs/report/data/vivado_ooc/rgb_to_grayscale/rgb_to_grayscale_ooc_utilization_synth.rpt
FRAME_COMPOSITOR core (OOC)	24.0	0.0	0.0	0.0	docs/report/data/vivado_ooc/frame_compositor_core/frame_compositor_core_ooc_utilization_synth.rpt
Integrated pipeline IP (OOC)	4189.0	4439.0	0.0	0.0	vivado/Zybo-Z7-10-Pcam-5C-hw.xpr/hw/hw.runs/system_AXI_RgbGrayBlurrSobe_0_0_synth_1/system_AXI_RgbGrayBlurrSobe_0_0_utilization_synth.rpt
Entire system (placed)	11944.0	15406.0	10.5	0.0	vivado/Zybo-Z7-10-Pcam-5C-hw.xpr/hw/hw.runs/impl_1/system_wrapper_utilization_placed.rpt

Table 7: Resource utilization extracted from Vivado synthesis/implementation reports.

Module	LUT share vs system	FF share vs system
RGB_TO_GRAYSCALE (AXI OOC)	0.402%	0.195%
FRAME_COMPOSITOR core (OOC)	0.201%	0.0%
Integrated pipeline IP (OOC)	 35.072%	 28.813%

Table 8: Relative LUT/FF share versus placed `system_wrapper` (CSV-driven bars rendered directly in Typst).

Observed snapshot in this revision:

- RGB_TO_GRAYSCALE (AXI OOC) and FRAME_COMPOSITOR core (OOC) are lightweight compared to full system placement,
- the integrated pipeline IP (`system_AXI_RgbGrayBlurrSobe_0_0`) dominates module-local logic among the measured design blocks,
- BRAM usage is concentrated outside these two small OOC component cuts in this snapshot.

9. Risks and Open Gaps

Open issues tracked in `.codex/ISSUES.md` that still impact closure quality:

- click-detector expectation drift in a legacy-aligned test target,
- runtime sensitivity for full-size pipeline regressions,
- potential prefill deadlock scenarios requiring dedicated non-passthrough stress tests,
- synthesis portability sensitivity from `ShiftRamChain` dependency on packaged `c_shift_ram_0`,
- process bootstrap gaps (missing `.codex/Questions.md` and missing local `fpga-vivado-vitis-structure` skill manifest/script assets).

These are closure and reproducibility risks; they do not invalidate the passing focused evidence in this revision, but they should be resolved before final publication.

10. Next Steps

1. Keep report source modular (`sections/*.typ`) and maintain command-backed evidence updates per revision.
2. Extend stress tests for overlay-mode prefill behavior with explicit deadlock-detection assertions.
3. Re-run full-pipeline regression with bounded wall-time budget and archive final artifact metrics.
4. Stabilize bootstrap/process assets referenced by AGENTS so required workflows are directly executable.

References

- [1] AMD, “AXI4-Stream Video IP and System Design Guide (UG934).” [Online]. Available: https://docs.amd.com/r/en-US/ug934_axi_videoIP
- [2] cocotb contributors, “cocotb Documentation: Writing Testbenches.” [Online]. Available: https://docs.cocotb.org/en/stable/writing_testbenches.html
- [3] cocotb contributors, “cocotb Documentation: The Timing Model.” [Online]. Available: https://docs.cocotb.org/en/stable/timing_model.html
- [4] Digilent, “Zybo Z7 Reference and Getting Started.” [Online]. Available: <https://digilent.com/reference/programmable-logic/zybo-z7/start>
- [5] Digilent, “Pcam 5C Reference and Getting Started.” [Online]. Available: <https://digilent.com/reference/add-ons/pcam-5c/start>
- [6] A. Forencich, “cocotbext-axi.” [Online]. Available: <https://github.com/alexforencich/cocotbext-axi>